Phase 4 – Validation & Pipeline Engineering

CampusBite Software Project

May 18, 2026

Contents

1 Introduction

CampusBite is a software platform developed to simplify food ordering inside university campuses. The system allows students to browse meals, place food orders, and manage transactions digitally through a web-based interface.

This phase focuses on software validation, automated testing, and pipeline engineering. The project applies the Testing Pyramid strategy, Playwright automation testing, and the Page Object Model (POM) architecture to improve software quality and maintainability.

2 Project Repository

The project source code is hosted on GitHub:

`https://github.com/MohamedIbrahim120230230/CampusBite/tree/main`

The repository contains the frontend application, backend services, configuration files, and testing resources used during the development process.

3 Testing Pyramid

The CampusBite project follows the Testing Pyramid methodology to ensure software reliability and maintainability.

3.1 Testing Pyramid Ratio

The following testing distribution was applied:

- 70% Unit Testing
- 20% Integration Testing
- 10% End-to-End (E2E) Testing

E2E Testing (10%)
Integration Testing (20%)
Unit Testing (70%)

3.2 Unit Testing

Unit testing was applied to small independent modules of the CampusBite application. These tests verified that each function behaved correctly in isolation.

Examples of unit-tested components include:

- User authentication validation
- Menu item processing
- Order calculation functions

- Input form validation
- Price and quantity calculations

Unit testing improves code quality by detecting bugs early during development.

3.3 Integration Testing

Integration testing verified communication between multiple modules inside the system.

Examples include:

- Frontend communication with backend APIs
- Database interaction with the order management system
- Authentication services linked with user accounts
- Checkout workflow with payment processing

These tests ensured that connected components operated correctly together.

3.4 End-to-End Testing

End-to-End testing simulated real user behavior across the complete CampusBite platform.

The following workflows were tested:

- User registration
- User login
- Browsing menu items
- Adding meals to cart
- Checkout and order placement

E2E testing verified that the entire system functioned properly from the user perspective.

4 Playwright Automation

Playwright was used to automate browser-based testing for CampusBite.

The project implemented the Page Object Model (POM) architecture to improve test readability, scalability, and maintainability.

4.1 Gherkin Scenarios

The following scenarios were written using Gherkin syntax:

```
Feature: User Login

Scenario: Successful Login
Given the user is on the login page
When the user enters valid credentials
Then the dashboard should appear

Scenario: Food Ordering
Given the user is logged into CampusBite
When the user selects a meal
And proceeds to checkout
Then the order should be successfully placed
```

4.2 Playwright Test Scripts

The Gherkin scenarios were converted into executable Playwright scripts.

Example Playwright automation:

```
import { test, expect } from '@playwright/test';

test('successful login', async ({ page }) => {
    await page.goto('http://localhost:3000/login');

    await page.fill('#email', 'student@test.com');
    await page.fill('#password', '123456');

    await page.click('button[type="submit"]');

    await expect(page).toHaveURL(/dashboard/);
});
```

4.3 Page Object Model (POM)

The Page Object Model separates page actions and page locators from the test scripts.

Advantages of POM include:

- Improved code organization
- Easier maintenance
- Reusable test components
- Better scalability for large projects

Project structure:

```
pages/
            LoginPage.js
            MenuPage.js
            CheckoutPage.js

tests/
            login.spec.js
            order.spec.js
```

Example Page Object:

```
export class LoginPage {
    constructor(page) {
        this.page = page;
    }

    async login(email, password) {
        await this.page.fill('#email', email);
        await this.page.fill('#password', password);
        await this.page.click('button[type="submit"]');
    }
}
```

5 Verification vs Validation

5.1 Verification

Verification ensures that the software was developed correctly according to technical specifications and design requirements.

Examples of verification activities in CampusBite include:

- Unit testing
- Integration testing
- Code reviews
- Static analysis

Verification answers the question:

"Are we building the product correctly?"

5.2 Validation

Validation ensures that the final software satisfies user requirements and solves the intended problem.

Examples of validation activities include:

- End-to-End testing
- User Acceptance Testing (UAT)

- Functional testing
- User feedback evaluation

Validation answers the question:

"Are we building the correct product?"

5.3 Comparison Between Verification and Validation

Verification	Validation
Checks correctness of implementation	Checks whether user needs are satisfied
Developer-focused process	User-focused process
Performed during development	Performed after functionality is completed
Includes unit and integration tests	Includes acceptance and usability testing

6 CI/CD Pipeline Engineering

A Continuous Integration and Continuous Deployment (CI/CD) workflow was considered during the development of CampusBite.

The pipeline process includes:

1. Code push to GitHub repository
2. Automatic dependency installation
3. Running automated Playwright tests
4. Validation of build success
5. Deployment preparation

Example GitHub Actions workflow:

```
name: Playwright Tests

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Install dependencies
      run: npm install
```

```
16
17      - name: Run Playwright tests
18        run: npx playwright test
```

7 Evidence and Results

The following results were achieved during testing:

- Successful execution of unit tests
- Verified frontend-backend integration
- Automated Playwright test execution
- Successful simulation of user workflows
- Improved reliability and maintainability through POM

The automated testing process reduced manual testing effort and improved overall software quality.

8 Conclusion

This phase successfully implemented validation and pipeline engineering practices for the CampusBite project.

The Testing Pyramid approach ensured balanced testing coverage, while Playwright automation improved testing efficiency. The Page Object Model enhanced maintainability and organization of automated tests.

Verification and validation activities confirmed both technical correctness and user satisfaction requirements. Overall, the project achieved a structured and reliable testing workflow suitable for modern software engineering practices.